

从 AI 提效到 AI Native 组织

执行系统、组织记忆与责任机制的共同改造

辛海洋 · 2026年8月31日 · 可可乐博

摘要

AI 系统正在降低信息检索、内容生成、软件开发和流程执行的边际成本。组织由此获得了扩大工作吞吐的条件，但执行速度提高并不自动转化为更可靠的产品判断或工程结果。任务所需的组织上下文是否完整，Agent 的行动是否受到权限与质量门槛约束，每次运行是否留下可检查并可用于后续工作的结果，开始成为更具决定性的条件。

本文综合 Uber Engineering 关于 Software Factory 的公开案例、Ashwin Gopinath 关于 Company Memory 的系列论述，以及可可乐博现阶段的实践框架，提出一条连续的组织改造路径：个人工具使用、工作流沉淀、Agent 托管执行、共享组织状态、基于结果的持续进化。该路径的重点不在 Agent 数量或自动化比例，而在组织能否将证据、判断、执行与结果建立为同一条可追溯状态链，并在明确的人类责任下持续运行。

本文建议以 Context、Harness、Loop 作为 AI Native 团队的基础结构。Context 提供完成任务所需的事实、决策原因和历史状态；Harness 提供工具、权限、预算、测试、审批和停止条件；Loop 负责执行、验证、人工修正、结果写回、版本更新和撤销。人的职责相应集中于目标定义、证据判断、例外处理和结果责任。

一、组织改造开始于工作单位的变化

在个人使用阶段，AI 系统主要承担搜索、整理、写作、分析和编码。其价值通常表现为某个步骤耗时下降，但完整任务仍由人临时拆解、组织上下文、选择工具并检查结果。这类使用可以形成明显的个人效率差异，却不必然形成稳定的组织能力。人员变化、任务环境变化或模型升级，都可能使原有做法失效。

组织级转型要求工作的基本单位从一次对话或一次生成，转向可重复、可验证的工作流。每条工作流需要明确目标、输入、输出、工具、权限、验收、异常处理和负责人。只有这些条件被显式化，Agent 才可能从个人助手转变为受治理的执行单元。

Uber Engineering 将 Agent 使用划分为个人会话、带 Skill 的会话、通用 Agent 和专用 Managed Agent 等层级。其公开文章强调，专用 Agent 需要围绕真实工作建立 Benchmark，并同时衡量结果质量、可靠性、延迟和单位结果成本。代码审查、CI 修复、告警处理和维护任务之所以适合逐步托管，是因为它们具有相对稳定的边界、可观察的结果和明确的升级路径。

Uber 披露的使用规模和成本变化属于企业自述，不能直接外推为其他组织的预期收益。该案例更可靠的启发是：Agent 需要被当作生产系统管理，而不是作为个人工具采购。模型只是运行变量，真实任务评测、统一运行环境、权限控制、运行记录和人工升级共同构成组织能力。

二、执行系统的上限取决于组织上下文

代码库、PRD、知识库和会议纪要保存了大量组织信息，但它们通常不能完全回答一项工作为何形成当前状态。架构决策可能没有记录被否决方案，产品需求可能省略客户承诺，流程文档可能没有包含真实例外，测试用例也未必说明某个历史客户或合规条件为何不能被破坏。

Ashwin Gopinath 将这一缺口描述为组织记忆问题。他区分了三类相互关联的记忆：

1. 事实记忆：发生了什么，证据在哪里，由谁负责，何时变化；
2. 互动与推理记忆：为什么这样判断，存在什么异议，哪些假设和问题尚未关闭；

3. 行动记忆：什么状态变化应触发行动，真实路径如何运行，谁批准或修正，结果是否符合预期。

这一区分说明，组织记忆不能等同于全文检索或会议摘要。Agent 完成任务所需的通常不是更多文本，而是能够区分来源事实、产品解释、未验证假设、已批准决策、条件性承诺和实际结果。没有这些区分，检索系统可能为 Agent 提供大量相关材料，却无法说明哪一项信息仍然有效，哪一项仅代表历史意见，哪一项决定了当前行动权限。

组织记忆也不应由每个工具分别维护。如果会议工具、代码 Agent、CRM 和项目系统各自形成局部记忆，公司仍然会产生相互冲突的局部真相。较为稳妥的做法是建立公司控制的共享状态，使其可检查、可纠正、可版本化、可授权并可迁移；不同角色可以从产品、研发、运营或管理视角读取同一组底层证据。

这一架构判断来自作者及其创业项目的公开论述，目前并非经过独立评测的普遍结论。其适用价值主要在于指出一个可被实际检验的问题：当 Agent 输出在评审中被拒绝时，失败究竟来自模型能力不足，还是来自组织没有向系统提供必要的决策原因和约束。

三、Context、Harness、Loop 构成共同基础

综合两组材料，可以将 AI Native 团队的基础结构概括为 Context、Harness 和 Loop。

1. Context：系统基于什么理解任务

Context 不只是文档集合，而是完成当前任务所需的受控状态。它至少应包含：

- 事实、来源、版本、有效期和访问权限；
- 用户或客户证据、产品解释和反例；
- 假设、异议、决策、承诺及其责任人；
- 架构、数据、安全、兼容性和合规约束；
- 历史执行、失败路径、人工修正和业务结果。

Context 的质量取决于来源和关系，而不是材料数量。第一阶段无需建设全公司大图谱，可以从一条产品到研发的真实链路建立小型、可核查的状态模型。

2. Harness：系统如何行动并受到约束

Harness 负责把模型能力组织为可治理的执行环境，包括 Agent、Skill、Tool、身份、权限、预算、超时、测试、日志、审批和回退。其目的既是提供行动能力，也是限制系统在证据不足或风险过高时继续行动。

每条工作流应显式定义以下行为：自动执行、提出候选方案、请求审批、等待、拒绝行动、升级和回滚。技术上能够调用某项工具，并不代表组织上已经授权该项行动。

3. Loop：一次执行如何转化为后续能力

Loop 将运行结果纳入组织学习。成功案例可以成为经过验证的先例，失败案例进入 Benchmark 和风险记忆，人工修正形成改进信号，业务结果用于判断工作流是否只是技术完成，还是实现了预期目标。

Skill 和 Workflow 的更新不宜自动成为组织规则。系统可以根据运行轨迹生成候选版本，但应经过真实任务回放、版本比较和具名批准，再进行受控发布。若新版本造成质量下降或权限风险，应能够撤销。

四、产品团队需要管理可追溯的判断状态

产品团队的主要产物不应只被理解为 PRD。PRD 是某一时点的表达形式，底层需要保留从证据到结果的判断状态：

原始证据 → 产品解释 → 异议或反例 → 假设 → 决策 → 承诺 → 行动 → 结果

这条状态链可以减少三类问题。第一，访谈摘要或高频反馈被直接写成确定需求；第二，新版 PRD 覆盖了旧决策的原因和未解决异议；第三，功能上线后没有回到原始假设，导致交付完成被误认为问题已经解决。

AI 系统可用于整理证据、定位冲突、生成候选状态、检查 PRD 缺口和汇总实验结果。候选决策、承诺、责任人和假设应由具名负责人确认后进入共享状态。涉及敏感交流、个人评价或未经授权的原始材料时，系统应保留分层权限，不以建立公司记忆为由扩大可见范围。

产品团队的改造重点包括：

- 将会议总结升级为决策、假设、异议、承诺和开放问题的候选提取；
- 为重要产品判断保留证据来源、样本范围、反例和失效条件；
- 使新证据能够定位其影响的假设、路线图决策和客户承诺；
- 为发布设置观察窗口、目标指标、反指标和撤回条件；
- 将发布结果写回判断状态，而不是停留在复盘文档。

五、研发团队需要管理可验证的执行状态

研发工作不应只留下代码 diff 和测试通过记录。中高风险任务还需要保存目标、约束、方案、被否决路径、验证环境、人工判断、发布结果和恢复方式。相应的执行链可以表示为：

目标 → 约束 → 方案 → 变更 → 测试 → 固定证据 → 批准 → 发布 → 结果写回

研发 Agent 在读取代码之外，还应获得与当前任务直接相关的 Decision Context 和 Failure Context，包括架构决策、历史事故、失败迁移、客户兼容性、安全要求和未验证假设。合并后需要写回能力、约束、风险或技术债的变化，并说明哪些文档、测试和产品承诺需要同步更新。

异常路径尤其需要结构化保存。Preview 通过但生产失败、依赖人工操作恢复、特定客户环境需要兼容处理等情况，不应只停留在聊天记录。它们应带有来源、版本、适用范围和失效条件，供下一次执行读取。

适合首批托管的研发工作包括 PR 首轮审查、CI 失败归因、候选修复、E2E 与视觉验证、固定 SHA 证据包和发布前检查。自动修改生产数据、扩大权限、处理不可逆迁移或直接发布外部承诺，不宜作为早期自动化目标。

六、产品与研发需要共享工作契约

产品判断和研发执行之间需要一个共同的 Product-to-Engineering Work Contract。该契约至少包含：

- 目标结果与验收标准；
- 用户或业务证据及其适用边界；

- 尚未验证的假设、已知异议和非目标；
- 技术、数据、安全和兼容性约束；
- Outcome Owner、执行负责人和 Approver；
- 验证环境、发布条件和回退机制；
- 结果观察窗口与写回责任。

该契约的作用不是增加文档负担，而是减少在产品、设计、研发、测试和管理之间反复重建上下文。Agent 可以帮助检查契约完整性、定位矛盾和生成执行计划，但不能自行补写未经确认的业务目标。

七、组织角色从执行分工转向责任分工

AI Native 团队并不取消专业分工。随着执行成本下降，目标、证据、架构、风险和结果判断的重要性上升。建议明确五类责任：

- Outcome Owner：定义目标、验收和停止条件，并对最终决定负责；
- Workflow Steward / AIBP：识别重复协调成本，维护工作流、Benchmark 和版本；
- Agent Platform Owner：负责运行环境、身份、权限、日志、成本和恢复；
- Evaluation & Safety Owner：维护反例、质量门槛、数据边界和发布检查；
- Human Reviewer：处理低置信、冲突、高风险和外部承诺。

小团队可以由一人承担多个角色，但责任类型不能消失。同一个 Agent 不应同时生成、验收并批准自己的结果；同一位人类负责人也不应在缺乏独立证据的情况下，将形式上的流程完成视为结果成立。

八、衡量标准从使用量转向可接受结果

Agent 数量、调用次数、Token 总量和 AI 参与率可以反映使用情况，但不能单独证明组织价值。建议将北极星指标定义为：

每单位人类监督时间产生的、通过验收的工作结果。

研发团队可进一步观察一次验收率、review minutes / merged PR、返工、revert、生产缺陷、cost / accepted PR、证据完整率和恢复成功率。产品团队可观察从证据到批准 PRD 的时间、假设与证据链接率、开发开始后的需求变更、实验周期、无负责人承诺数和发布后结果写回率。

跨团队还需要关注上下文重建时间、相同问题重复调查次数、冲突状态解决时间、人工审批队列和越权行动。随着 Agent 行动数量增长，即使单次错误率下降，错误绝对数量和人工复核负担仍可能增加。因此，提升自动化权限之前，需要同时确认结果质量和责任容量没有恶化。

九、建议的改造顺序

M0：共同状态语言与基线（第1-2周）

产品与研发共同定义 Evidence、Assumption、Dissent、Decision、Commitment、Trigger、Action、Outcome 和 Owner，选择两条工作流测量上下文重建时间、返工和人工复核基线。

M1：只读状态链（第3-4周）

选择一个真实功能，建立从用户证据、产品判断、开发任务到固定 SHA 验收和发布结果的可追溯链。Agent 仅用于检索和生成候选状态，不自动写入公司事实。

M2：人工确认后的状态写回（第2月）

会议、PR 和发布后生成候选状态变化，由具名 Owner 确认后写入。系统保留来源、版本、异议、权限和失效条件。

M3：Trigger 影子运行（第3月）

对承诺没有 Owner、假设被新证据挑战、历史失败模式重现、发布条件满足或失效等情况进行只读告警。通过历史回放测量误报、漏报和人工处理成本。

M4：单条受控 Action Loop（第4-6月）

选择一条低风险链路，从已批准需求进入开发、验证、人工发布批准和结果写回。工作流必须具备 wait、abstain、escalate 和 rollback，并通过连续运行和回退演练后再扩大权限。

M5：可迁移与规模化（第7-12月）

扩展第二条跨团队工作流，并进行模型或 Agent Harness 替换。核心 Context、Benchmark、Skill 版本、批准和运行记录应能够完整导出；更换模型后，真实任务评测不能出现不可接受的退化。

十、当前最值得实施的纵向切口

第一阶段不宜建设全公司知识图谱，也不宜追求全面自动化。建议选择以下真实链路：

用户或教师证据 → 产品判断 → PRD → 开发任务 → PR → 固定 SHA 验收 → 人工发布批准 → 结果写回

该切口覆盖产品与研发的主要交接点，并且能够检验以下条件：事实与判断是否区分，决策原因是否可追溯，Agent 是否获得必要上下文，执行是否具备权限和停止条件，批准是否具名并有证据依据，发布结果是否反向更新产品判断和工程经验。

首期成果不应以新增 Agent 数量表述，而应以一条端到端状态链是否可检查、可纠正、可恢复和可复现来判断。

结语

AI Native 转型可以被理解为一连续的能力建设路径：个人工具使用、工作流复用、Agent 托管执行、组织状态共享、基于结果的持续进化。

组织由此改变的不只是任务执行速度。更重要的变化是，事实、判断、行动和结果开始进入同一个可治理系统。人仍然承担目标、边界、例外和结果责任，Agent 则在明确 Context 和 Harness 的条件下执行可验证工作。每次运行留下的证据和修正进入下一次任务，使工作不再依赖关键成员临场重建全部上下文。

判断转型是否成立，应观察组织能否在关键成员缺席时重复完成同类工作，是否能够说明某项结果为何可信，是否能够在失败后恢复，以及新证据出现时能否定位受到影响的决策和承诺。这些条件比模型使用率或自动化比例更接近 AI Native 组织的实质。

主要资料

1. Uber Engineering. *Running a Software Factory Efficiently at Uber Scale*. 2026-08-27.
<https://www.uber.com/us/en/blog/efficient-software-factory/>
2. Uber Engineering. *uReview: Scalable, Trustworthy GenAI for Code Review at Uber*. 2025-08-12.
<https://www.uber.com/us/en/blog/ureview/>
3. Uber Engineering. *Lessons from Building a First-Pass AI PRD Reviewer at Uber*. 2026-05-12.
<https://www.uber.com/us/en/blog/first-pass-prd/>
4. Ashwin Gopinath. *Company Brain: Why Most Companies Have Data But No Memory*. 2026-05-04.
<https://nanothoughts.substack.com/p/company-brain-why-most-companies>
5. Ashwin Gopinath. *Company Brain, Part 3: Interaction Memory*. 2026-05-06.
<https://nanothoughts.substack.com/p/company-brain-part-3-interaction>
6. Ashwin Gopinath. *Company Brain, Part 4: Action Memory*. 2026-05-07.
<https://nanothoughts.substack.com/p/company-brain-part-4-action-memory>
7. Ashwin Gopinath. *Memory Is State, Not a Service*. 2026-05-08.
<https://nanothoughts.substack.com/p/memory-is-state-not-a-service>
8. Ashwin Gopinath. *Cheap Intelligence Makes Accountability Expensive*. 2026-05-14.
<https://nanothoughts.substack.com/p/cheap-intelligence-makes-accountability>
9. Ashwin Gopinath. *Rent the Intelligence. Own the Context.* 2026-05-18.
<https://nanothoughts.substack.com/p/rent-the-intelligence-own-the-context>

资料边界

Uber 的规模、质量和成本数据属于企业自述，公开资料未提供完整口径或外部审计。Ashwin Gopinath 同时是 Sentra 创业者，其 Company Brain 架构具有明确的产品立场。本文提出的五阶段演进、团队责任和实施顺序属于基于上述材料及现阶段实践作出的分析与建议，不构成已验证的普遍效果结论。